

INTERVIEW PREPARATION SERIES

Java Exception Handling

Interview Q&A — 40 Questions

This document covers **40 interview questions** spanning the Throwable hierarchy, Error vs Exception, checked vs unchecked exceptions, try-catch-finally, try-with-resources, custom exceptions, exception chaining, re-throwing, multi-catch, suppressed exceptions, exception handling in streams and lambdas, CompletableFuture error handling, and Java best practices. Designed for Java developers and full-stack engineers.

Throwable Hierarchy & Fundamentals

QUESTION 01

What is the exception hierarchy in Java? Explain the relationship between Throwable, Error, Exception, and RuntimeException.

ANSWER

```
//          Throwable
//          /      \
//         Error    Exception
//        /      /   \
// StackOverflowError IOException RuntimeException
// OutOfMemoryError   SQLException /      \
// VirtualMachineError ...      NullPointerException
//                               IndexOutOfBoundsException
//                               IllegalArgumentException
```

- **Throwable:** the root class. Only Throwable and its subclasses can be thrown/caught.
- **Error:** serious JVM-level problems. **Should not be caught** — the JVM is in an unrecoverable state (StackOverflowError, OutOfMemoryError).
- **Exception:** application-level problems. Divided into checked (must handle) and unchecked (runtime).

- **RuntimeException:** subclass of Exception. **Unchecked** — compiler does not force handling. Represents programming bugs (NullPointerException, ClassCastException).

QUESTION 02

What is the difference between checked and unchecked exceptions in Java? Why does Java have both?

ANSWER

Checked exceptions: subclasses of Exception (but not RuntimeException). The compiler **forces** you to either catch them or declare them with **throws**.

Examples: IOException, SQLException, FileNotFoundException.

Unchecked exceptions: subclasses of RuntimeException. The compiler does **not** enforce handling. Examples: NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException.

Why both exist:

- **Checked:** represent **foreseeable, recoverable failures** (file not found, network down). The caller should be forced to think about recovery.
- **Unchecked:** represent **programming bugs** (null dereference, bad cast). Forcing every method to declare these would make code unreadable.

Modern debate: many frameworks (Spring, Hibernate) wrap checked exceptions in unchecked ones. Some argue checked exceptions add verbosity without proportional safety. Java remains the only mainstream language that has them.

QUESTION 03

What is **Error** in Java? Name common Error types and explain why you should not catch them.

ANSWER

- **StackOverflowError:** call stack exhausted, usually from infinite recursion
- **OutOfMemoryError:** JVM heap is full, cannot allocate new objects
- **VirtualMachineError:** JVM internal failure
- **NoClassDefFoundError:** class was available at compile time but not at runtime

- **LinkageError:** class dependency conflicts

Why not catch: Errors indicate the JVM is in an **unrecoverable state**. Catching `OutOfMemoryError` and continuing means the JVM has no memory — any subsequent operation may fail unpredictably. The correct response is to let the JVM terminate, fix the root cause, and restart.

Exception: you **may** catch `Error` in specific cases: logging frameworks catch `Throwable` to ensure errors are recorded before shutdown. Servers may catch `OutOfMemoryError` to gracefully decline new requests.

QUESTION 04

What is the difference between `throw` and `throws` in Java?

ANSWER

throw: an **action** — it creates and throws an exception object from inside a method:

```
public void withdraw(double amount) {
    if (amount > balance) {
        throw new InsufficientFundsException(balance, amount);
    }
    balance -= amount;
}
```

throws: a **declaration** — it tells the caller that this method might throw specific checked exceptions:

```
public void readFile(String path) throws IOException, FileNotFoundException
{
    FileReader reader = new FileReader(path); // may throw
    // ...
}
```

Key distinction: `throw` is used inside method bodies. `throws` is part of the method signature. You `throw` an instance; you `throws` a class.

QUESTION 05

How does `try-catch-finally` work in Java? What are the execution guarantees of the `finally` block?

ANSWER

```
try {
    Connection conn = DriverManager.getConnection(url);
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM users");
} catch (SQLException e) {
    logger.error("Database error: " + e.getMessage(), e);
    throw new ServiceException("Failed to query users", e);
} finally {
    // ALWAYS executes: success, exception, or even return
    if (conn != null) conn.close();
}
```

finally guarantees:

- Executes whether the try block succeeds, throws, or returns
- Executes even if catch block throws or returns
- Only exception: `System.exit()` or JVM crash prevents finally

finally + return trap: if both try and finally have return statements, finally's return **wins**. This is a common interview trick question.

Advanced Exception Mechanics

QUESTION 06

What is try-with-resources in Java? How does it work with `AutoCloseable`, and why is it preferred over try-finally?

ANSWER

```
// try-with-resources (Java 7+)
try (Connection conn = DriverManager.getConnection(url);
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()) {
    while (rs.next()) {
        // process results
    }
} catch (SQLException e) {
    logger.error("Query failed", e);
}
// conn, stmt, rs are ALL automatically closed in reverse order
```

How it works: any class implementing `AutoCloseable` (or `Closeable`) can be declared in the try parentheses. The JVM calls `close()` on each resource in **reverse declaration order** when the block exits.

Why preferred:

- No resource leaks: `close()` is called even if an exception is thrown
- Suppressed exceptions: if `close()` also throws, it is added as a **suppressed exception** (accessible via `getSuppressed()`) instead of hiding the original
- Cleaner code: no nested try-finally blocks

QUESTION 07

What is exception chaining in Java? What is the `cause` of an exception, and how do you use it?

ANSWER

Exception chaining preserves the original exception (the **cause**) when wrapping it in a higher-level exception. This maintains the full error context for debugging.

```
public User findUser(int id) throws ServiceException {
    try {
        return userDao.findById(id);
    } catch (SQLException e) {
        // Wrap the low-level exception in a high-level one
        throw new ServiceException("Failed to find user " + id, e);
        //                                     ^^^ cause
    }
}

// Later, in the error handler:
catch (ServiceException e) {
    logger.error(e.getMessage());           // "Failed to find user 42"
    logger.error(e.getCause().getMessage()); // "Connection refused"
    e.printStackTrace(); // shows BOTH stack traces
}
```

All standard exception constructors accept a **Throwable** `cause` parameter.

Always chain — never throw a new exception without including the original cause, or you lose the root cause information.

QUESTION 08

What is re-throwing an exception? What are the different patterns for rethrowing, and when do you use each?

ANSWER

```
// Pattern 1: Rethrow the same exception (preserve stack trace)
try {
    riskyOperation();
} catch (IOException e) {
    logger.error("Operation failed", e);
    throw e; // rethrow as-is
}

// Pattern 2: Wrap and rethrow (translate to higher-level exception)
try {
    riskyOperation();
} catch (IOException e) {
    throw new ServiceException("Service unavailable", e); // chained
}

// Pattern 3: Rethrow with more context
try {
    processFile(path);
} catch (IOException e) {
    throw new IOException("Failed processing: " + path, e);
}

// Pattern 4: Selective rethrow
try {
    complexOperation();
} catch (RecoverableException e) {
    handleGracefully(e); // handle this one
} catch (FatalException e) {
    throw e; // rethrow this one
}
```

Golden rule: if you catch an exception just to log it, **always rethrow**. Catching and swallowing silently hides bugs.

QUESTION 09

What is multi-catch in Java? How does it work, and what are its restrictions?

ANSWER

```
// Before Java 7: repetitive catch blocks
try {
    parseAndProcess(input);
} catch (NumberFormatException e) {
    handleError(e);
}
```

```
} catch (ParseException e) {
    handleError(e); // same handling, duplicated code
}

// Java 7+: multi-catch
try {
    parseAndProcess(input);
} catch (NumberFormatException | ParseException e) {
    handleError(e); // single handler for both
}
```

Restrictions:

- The exception types in a multi-catch **cannot be related by inheritance**. `catch (IOException | FileNotFoundException e)` is a compile error because `FileNotFoundException` extends `IOException`.
- The variable `e` is implicitly **final** in a multi-catch — you cannot reassign it.
- The compiler infers the type as the **common supertype** of the caught exceptions.

QUESTION 10

What are suppressed exceptions in Java? When do they occur, and how do you access them?

ANSWER

Suppressed exceptions occur in try-with-resources when **both the try block and the close() method throw exceptions**. The `close()` exception is **suppressed** (not lost) and attached to the primary exception.

```
try (MyResource res = new MyResource()) {
    res.doWork(); // throws PrimaryException
} // res.close() also throws CloseException (suppressed)

catch (PrimaryException e) {
    System.out.println(e.getMessage()); // "Primary error"
    for (Throwable suppressed : e.getSuppressed()) {
        System.out.println(suppressed.getMessage()); // "Close error"
    }
}
```

Before try-with-resources, the `close()` exception would **replace** the original exception in a finally block — the primary error was lost. Suppressed exceptions solve this by preserving both.

Custom Exceptions & Best Practices

QUESTION 11

How do you create a custom exception in Java? What are the best practices for designing an exception hierarchy?

ANSWER

```
// Base application exception
public class AppException extends RuntimeException {
    private final String errorCode;

    public AppException(String message, String errorCode) {
        super(message);
        this.errorCode = errorCode;
    }

    public AppException(String message, String errorCode, Throwable cause)
    {
        super(message, cause); // ALWAYS include cause constructor
        this.errorCode = errorCode;
    }

    public String getErrorCode() { return errorCode; }
}

// Specific exceptions extend the base
public class UserNotFoundException extends AppException {
    public UserNotFoundException(long userId) {
        super("User not found: " + userId, "USER_NOT_FOUND");
    }
}

public class InsufficientFundsException extends AppException {
    private final BigDecimal balance;
    private final BigDecimal amount;
    // ... constructor with context fields
}
```

Best practices:

- Always provide constructors accepting a **Throwable cause**
- Include contextual data (userId, balance, amount) as fields
- Create a base exception for your module with an error code

- Extend RuntimeException for most custom exceptions (modern Java convention)

QUESTION 12

What are the common exception handling anti-patterns in Java that should be avoided?

ANSWER

```
// ANTI-PATTERN 1: Empty catch (swallowing exceptions)
try { riskyOp(); } catch (Exception e) { } // BUG: error silently ignored

// ANTI-PATTERN 2: Catching Exception/Throwable broadly
try { riskyOp(); } catch (Exception e) { log(e); }
// Catches NullPointerException, ClassCastException too!

// ANTI-PATTERN 3: Losing the cause
try { dao.query(); }
catch (SQLException e) {
    throw new ServiceException(e.getMessage()); // cause LOST!
    // CORRECT: throw new ServiceException("msg", e);
}

// ANTI-PATTERN 4: Using exceptions for control flow
try { int i = Integer.parseInt(input); }
catch (NumberFormatException e) { /* not a number */ }
// BETTER: validate before parsing

// ANTI-PATTERN 5: Logging and rethrowing (double logging)
catch (IOException e) {
    logger.error("Error", e); // logged here
    throw e; // AND logged by the caller
}
```

QUESTION 13

When should you use checked vs unchecked exceptions for your custom exceptions?

ANSWER

Use checked (extends Exception) when:

- The caller can and should recover from the error
- The failure is **expected and foreseeable** (network timeout, file not found)

- You want the compiler to force callers to handle it

Use unchecked (extends RuntimeException) when:

- The error represents a **programming bug** (invalid argument, illegal state)
- Recovery is not possible or meaningful at the call site
- Forcing every caller to catch it would add noise without value

Modern convention: Spring, Hibernate, and most modern Java frameworks use **unchecked exceptions exclusively**. The reasoning: if you cannot recover, forcing a catch clause adds verbosity without safety. If you can recover, you will write the catch even without the compiler forcing you.

QUESTION 14

How does exception handling interact with method overriding? What are the rules for exceptions in overridden methods?**ANSWER****Rules for overriding methods:**

- An overriding method can throw **fewer or narrower** checked exceptions than the parent
- An overriding method **cannot** throw **new or broader** checked exceptions
- Unchecked exceptions have no restrictions

```
class Parent {
    void process() throws IOException { }
}

class Child extends Parent {
    // OK: narrower exception
    void process() throws FileNotFoundException { }

    // OK: no exception at all
    // void process() { }

    // COMPILE ERROR: broader exception
    // void process() throws Exception { }

    // OK: unchecked exceptions are always allowed
    // void process() throws IllegalArgumentException { }
}
```

Reason: code written against the parent type expects at most IOException. If the child threw SQLException, the catch clause would miss it, violating the Liskov Substitution Principle.

QUESTION 15

What is `finally` vs `try-with-resources` for resource management? Show a comparison of both approaches.

ANSWER

```
// Old way: try-finally (error-prone)
Connection conn = null;
try {
    conn = DriverManager.getConnection(url);
    // use connection...
} catch (SQLException e) {
    logger.error("DB error", e);
} finally {
    if (conn != null) {
        try { conn.close(); }
        catch (SQLException e) { logger.error("Close failed", e); }
    }
}

// Modern way: try-with-resources (clean, safe)
try (Connection conn = DriverManager.getConnection(url)) {
    // use connection...
} catch (SQLException e) {
    logger.error("DB error", e);
}
// conn.close() called automatically, close exception suppressed
```

`try-with-resources` is **always preferred**: fewer lines, no null checks, no nested try-catch in finally, and proper suppressed exception handling.

QUESTION 16

What is `StackOverflowError`, and how does it differ from an `OutOfMemoryError`? How do you debug each?

ANSWER

- **StackOverflowError:** the call stack exceeds its limit (default ~512KB-1MB). Usually caused by infinite or very deep recursion.

- **OutOfMemoryError:** the JVM heap is exhausted. Caused by memory leaks, loading too much data, or insufficient heap allocation.

```
// StackOverflow: look at the stack trace for the repeating method
void infinite() { infinite(); } // StackOverflowError
// Fix: convert recursion to iteration, or increase stack: -Xss4m

// OutOfMemory: use heap dump analysis
// Run with: -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/
// Analyze with: Eclipse MAT, VisualVM, or jmap
// Fix: find the memory leak (unclosed connections, caches, event
listeners)
```

QUESTION 17

How do you handle exceptions in Java streams and lambdas? Why is it problematic?

ANSWER

Problem: functional interfaces (Function, Predicate, Consumer) do **not declare checked exceptions**. You cannot throw IOException from a lambda passed to `stream.map()`.

```
// COMPILER ERROR: lambda cannot throw checked exception
list.stream().map(path -> Files.readString(path)) // IOException!

// Solution 1: Wrap in unchecked exception
list.stream().map(path -> {
    try { return Files.readString(path); }
    catch (IOException e) { throw new UncheckedIOException(e); }
})

// Solution 2: Extract to a method with @SneakyThrows (Lombok)
// Solution 3: Use a wrapper functional interface
@FunctionalInterface
interface CheckedFunction<T, R> {
    R apply(T t) throws Exception;
}
```

This is one of the most criticized aspects of checked exceptions in Java — they interact poorly with lambdas and the Stream API.

QUESTION 18

What is the `UncheckedIOException` class? Why was it added in Java 8?

ANSWER

`UncheckedIOException` wraps an `IOException` as an unchecked exception. Added in Java 8 specifically to work with the Stream API and lambdas.

```
// Before: awkward try-catch inside lambda
stream.map(file -> {
    try { return Files.readString(file); }
    catch (IOException e) { throw new RuntimeException(e); } // loses type
});

// After: purpose-built wrapper
stream.map(file -> {
    try { return Files.readString(file); }
    catch (IOException e) { throw new UncheckedIOException(e); }
});

// Caller can catch and unwrap
try { processFiles(paths); }
catch (UncheckedIOException e) {
    IOException original = e.getCause(); // typed cause!
}
```

QUESTION 19

How does exception handling work in multi-threaded Java code? What happens when a thread throws an exception?

ANSWER

When a thread throws an unhandled exception, the thread **terminates silently** unless a handler is set.

```
// Set default handler for ALL uncaught exceptions
Thread.setDefaultUncaughtExceptionHandler((thread, throwable) -> {
    logger.error("Uncaught in " + thread.getName(), throwable);
});

// Set handler for a specific thread
thread.setUncaughtExceptionHandler((t, e) -> {
    logger.error("Thread " + t.getName() + " failed", e);
});

// ExecutorService: exceptions are captured in the Future
```

```
Future<?> future = executor.submit(() -> riskyOperation());
try {
    future.get(); // throws ExecutionException wrapping the original
} catch (ExecutionException e) {
    Throwable cause = e.getCause(); // the original exception
}
```

With CompletableFuture: `exceptionally()` and `handle()` provide functional error handling: `future.exceptionally(ex -> defaultValue)`.

QUESTION 20

What are `assert` statements in Java? How do they relate to exception handling?

ANSWER

```
// Assert: verify assumptions during development
assert age >= 0 : "Age cannot be negative: " + age;
// Throws AssertionError if condition is false
// ONLY when assertions are enabled: java -ea MyApp
```

Key points:

- Assertions are **disabled by default** in production. Enable with `-ea` flag
- `AssertionError` extends `Error`, not `Exception`
- Use assertions for internal invariants, not for input validation
- NEVER use assertions to validate user input or method arguments — use `IllegalArgumentException` instead

When to use: verify postconditions, loop invariants, and unreachable code (`assert false : "Should not reach here"`). When to avoid: validating external input, enforcing business rules.

QUESTION 21

What is the difference between `final`, `finally`, and `finalize` in Java?

ANSWER

- **final:** keyword for constants (`final int x = 5`), preventing inheritance (`final class`), and preventing method overriding (`final void method()`)
- **finally:** exception handling block that always executes after try/catch

- **finalize():** method called by the GC before an object is collected. **Deprecated since Java 9.** Unreliable (no guarantee when or if it runs). Use `Cleaner` or `try-with-resources` instead

QUESTION 22

How does the `throws` clause interact with interfaces? If an interface method declares `throws IOException`, what are the rules for implementations?

ANSWER

```
interface DataReader {
    String read() throws IOException;
}

// Implementation can throw the SAME exception
class FileReader implements DataReader {
    public String read() throws IOException { ... }
}

// Implementation can throw a NARROWER exception
class CsvReader implements DataReader {
    public String read() throws FileNotFoundException { ... }
}

// Implementation can throw NO exception
class CacheReader implements DataReader {
    public String read() { ... } // no throws at all
}

// Implementation CANNOT throw BROADER exceptions
// COMPILE ERROR:
// class BadReader implements DataReader {
//     public String read() throws Exception { ... }
// }
```

QUESTION 23

What is `ExceptionInInitializerError`? When does it occur?

ANSWER

Thrown when a static initializer block or static field initialization throws an exception:

```
class Config {
    static final Properties props;
    static {
        try {
            props = loadProperties("/config.properties");
        } catch (IOException e) {
            throw new ExceptionInInitializerError(e);
        }
    }
}
```

This wraps the original exception in an Error because static initialization happens during class loading — there is no application code to catch exceptions yet. If this Error occurs, the class is marked as **unusable** and subsequent use throws `NoClassDefFoundError`.

QUESTION 24

How do you handle exceptions in constructors? What happens if a constructor throws an exception?

ANSWER

If a constructor throws, the object is **never fully created**. The reference is null; any resources allocated before the exception are leaked unless handled.

```
// PROBLEM: resource leak if second resource fails
public Connection(String host, int port) throws IOException {
    this.socket = new Socket(host, port);    // succeeds
    this.stream = socket.getInputStream();    // fails! socket leaked!
}

// FIX: try-catch in constructor, close on failure
public Connection(String host, int port) throws IOException {
    try {
        this.socket = new Socket(host, port);
        this.stream = socket.getInputStream();
    } catch (IOException e) {
        if (socket != null) socket.close();
        throw e; // rethrow after cleanup
    }
}
```

Best practice: use factory methods that return a fully-constructed object, wrapping the constructor in try-with-resources.

QUESTION 25

What is the `ClassCastException`, and how is it related to generics and type erasure?

ANSWER

```
// ClassCastException at runtime
Object obj = "hello";
Integer num = (Integer) obj; // ClassCastException!

// Hidden by raw types (bypasses generics)
List list = new ArrayList();
list.add("not a number");
int value = (Integer) list.get(0); // ClassCastException at runtime

// Generics PREVENT this at compile time
List<Integer> safeList = new ArrayList<>();
// safeList.add("not a number"); // COMPILER ERROR
```

Type erasure: Java generics are erased at compile time. `List<String>` becomes `List` at runtime. If you bypass generics (raw types, unchecked casts), `ClassCastException` can occur.

QUESTION 26

How does Spring Framework handle exceptions? What is `@ExceptionHandler` and `@ControllerAdvice`?

ANSWER

```
// Global exception handler for REST APIs
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(UserNotFoundException e) {
        return new ErrorResponse(e.getErrorCode(), e.getMessage());
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleGeneral(Exception e) {
        logger.error("Unhandled exception", e);
    }
}
```

```
        return new ErrorResponse("INTERNAL_ERROR", "Something went wrong");
    }
}
```

Spring converts exceptions to HTTP responses automatically. `@ControllerAdvice` applies globally across all controllers. This eliminates try-catch blocks in individual controller methods.

QUESTION 27

What is the `NullPointerException`, and how do you prevent it using `Optional` and defensive coding?

ANSWER

```
// NullPointerException: the most common Java exception
String name = user.getAddress().getCity().toUpperCase();
// If user, address, or city is null → NPE

// Fix 1: Defensive null checks
if (user != null && user.getAddress() != null &&
    user.getAddress().getCity() != null) {
    name = user.getAddress().getCity().toUpperCase();
}

// Fix 2: Optional (Java 8+) – cleaner
String name = Optional.ofNullable(user)
    .map(User::getAddress)
    .map(Address::getCity)
    .map(String::toUpperCase)
    .orElse("Unknown");
```

Prevention: use `@NonNull` annotations, return `Optional` instead of null, use `Objects.requireNonNull()` at method entry points.

QUESTION 28

How do you test exception handling in unit tests with JUnit?

ANSWER

```
// JUnit 5: assertThrows
@Test
void withdrawShouldThrowWhenInsufficientFunds() {
    Account account = new Account(100);
```

```
InsufficientFundsException ex = assertThrows(
    InsufficientFundsException.class,
    () -> account.withdraw(200)
);

assertEquals(100, ex.getBalance());
assertEquals(200, ex.getAmount());
assertTrue(ex.getMessage().contains("100"));
}

// Verify NO exception is thrown
@Test
void depositShouldNotThrow() {
    assertDoesNotThrow(() -> account.deposit(50));
}
```

QUESTION 29

List the most common `RuntimeException` subclasses and explain when each occurs.

ANSWER

- **NullPointerException:** calling a method or accessing a field on a null reference
- **IndexOutOfBoundsException:** array or list access with invalid index
- **ClassCastException:** invalid type cast
- **IllegalArgumentException:** method receives an invalid argument
- **IllegalStateException:** method called at the wrong time (e.g., `iterator.remove()` before `next()`)
- **NumberFormatException:** string cannot be parsed to a number
- **UnsupportedOperationException:** operation not supported (e.g., modifying an unmodifiable list)
- **ConcurrentModificationException:** collection modified during iteration
- **ArithmeticException:** illegal arithmetic (e.g., integer division by zero)
- **StackOverflowError:** technically an Error, not RuntimeException, but often confused

QUESTION 30

What is the performance impact of exception handling in Java? Are exceptions expensive?

ANSWER

Creating and throwing an exception is expensive because:

- **Stack trace generation:** `fillInStackTrace()` walks the entire call stack and records every frame. This is the primary cost.
- **Object allocation:** creating the exception object on the heap

How expensive: throwing an exception is ~100-1000x slower than returning a value. Creating 10,000 exceptions per second is noticeable.

Optimization: for exceptions used as control flow (hot paths), override `fillInStackTrace()` to return `this` (skip stack trace). Or use error codes/Optional instead of exceptions for expected failures.

Rule: exceptions are for **exceptional** situations. If 50% of calls throw, you have a design problem. Use return values (Optional, Result types) for expected failures.

QUESTION 31

What is `InterruptedException`, and how should you handle it correctly in multi-threaded code?

ANSWER

```
// InterruptedException: thread was interrupted while waiting/sleeping
try {
    Thread.sleep(5000);
} catch (InterruptedException e) {
    // WRONG: swallowing it
    // catch (InterruptedException e) { }

    // CORRECT: restore the interrupt flag
    Thread.currentThread().interrupt();
    throw new RuntimeException("Thread interrupted", e);
}
```

Why restore the interrupt: catching `InterruptedException` **clears the interrupt flag**. If you don't restore it, code higher up the call stack will not know the thread was interrupted.

QUESTION 32

How do you create an exception that carries structured error data for API responses?

ANSWER

```
public class ApiException extends RuntimeException {
    private final int statusCode;
    private final String errorCode;
    private final Map<String, String> details;

    public ApiException(int statusCode, String errorCode,
                       String message, Map<String, String> details) {
        super(message);
        this.statusCode = statusCode;
        this.errorCode = errorCode;
        this.details = details;
    }

    // Usage:
    // throw new ApiException(400, "VALIDATION_ERROR", "Invalid input",
    //      Map.of("email", "Invalid format", "age", "Must be positive"));
}
```

QUESTION 33

What is the difference between `e.getMessage()`, `e.toString()`, and `e.printStackTrace()`?

ANSWER

```
Exception e = new IllegalArgumentException("bad value");

e.getMessage();      // "bad value"
e.toString();        // "java.lang.IllegalArgumentException: bad value"
e.printStackTrace(); // full stack trace printed to System.err
```

For logging: pass the exception as the **second argument** to the logger: `logger.error("Operation failed", e)`. The logging framework handles the stack trace formatting. **Never** use `printStackTrace()` in production — use a proper logging framework.

QUESTION 34

How does exception handling work in Java's `CompletableFuture` for async operations?

ANSWER

```
CompletableFuture.supplyAsync(() -> fetchUser(id))
    .thenApply(user -> processUser(user))
    .exceptionally(ex -> {
        logger.error("Pipeline failed", ex);
        return defaultUser();
    })
    .thenAccept(result -> sendResponse(result));

// handle() processes both success and failure
CompletableFuture.supplyAsync(() -> fetchUser(id))
    .handle((result, ex) -> {
        if (ex != null) return defaultUser();
        return processUser(result);
    });
```

`exceptionally()` is the async equivalent of `catch`. `handle()` is like `try-catch-finally` combined. The exception type is always `CompletionException` wrapping the original.

QUESTION 35

What is a `try` with no `catch` — just `try-finally`? When is this pattern used?

ANSWER

```
// try-finally without catch: cleanup without handling
public void processWithLock() {
    lock.lock();
    try {
        doWork(); // any exception propagates to caller
    } finally {
        lock.unlock(); // lock is ALWAYS released
    }
}
```

Used when: you need cleanup (release a lock, close a resource) but do not want to **handle** the exception — let it propagate. The `finally` block guarantees cleanup regardless of what happens.

QUESTION 36**How do you handle exceptions in Java servlet filters and Spring interceptors?****ANSWER**

```
// Servlet filter: exception wrapping
@Override
public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
    throws IOException, ServletException {
    try {
        chain.doFilter(req, res);
    } catch (Exception e) {
        logger.error("Request failed", e);
        ((HttpServletResponse) res).sendError(500, "Internal Error");
    }
}
```

Spring interceptors use `@ControllerAdvice` which is cleaner. Filters are lower-level (Servlet API) and must handle exceptions within the filter itself.

QUESTION 37**What is `NoSuchElementException`? Where does it commonly occur?****ANSWER**

- `Iterator.next()` when there are no more elements
- `Scanner.nextInt()` when the input has no more tokens
- `Optional.get()` when the Optional is empty

```
// Common pitfall with Optional
Optional<User> user = findUser(id);
user.get(); // throws NoSuchElementException if empty!

// Correct usage
User u = user.orElseThrow(() -> new UserNotFoundException(id));
```

QUESTION 38**How do you implement a global exception handler for a Java microservice that returns structured JSON error responses?****ANSWER**

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorBody> handle(UserNotFoundException e) {
        return ResponseEntity.status(404).body(
            new ErrorBody("USER_NOT_FOUND", e.getMessage()));
    }

    @ExceptionHandler(ConstraintViolationException.class)
    public ResponseEntity<ErrorBody>
    handleValidation(ConstraintViolationException e) {
        Map<String, String> errors = e.getConstraintViolations().stream()
            .collect(Collectors.toMap(
                v -> v.getPropertyPath().toString(),
                ConstraintViolation::getMessage));
        return ResponseEntity.badRequest().body(
            new ErrorBody("VALIDATION_FAILED", "Invalid input", errors));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorBody> handleAll(Exception e) {
        logger.error("Unhandled exception", e);
        return ResponseEntity.status(500).body(
            new ErrorBody("INTERNAL_ERROR", "An error occurred"));
    }
}
```

QUESTION 39

What is `ClassNotFoundException` vs `NoClassDefFoundError`? How do you diagnose each?

ANSWER

- **`ClassNotFoundException`: checked exception.** Thrown when `Class.forName()` or `ClassLoader.loadClass()` cannot find the class. Usually means a missing JAR in the classpath.
- **`NoClassDefFoundError`: Error.** The class was available at compile time but not at runtime. Usually means a dependency was available during build but missing in the deployment (e.g., a JAR not included in the Docker image).

Diagnosis: check the classpath (`java -cp`), verify all JARs are included in the deployment artifact, check for version conflicts using `mvn dependency:tree` or Gradle's dependency report.

QUESTION 40

Summarize the 10 golden rules of exception handling in Java.

ANSWER

- **1.** Never swallow exceptions — at minimum, log them
 - **2.** Catch specific exceptions, not Exception or Throwable
 - **3.** Always chain the cause: `throw new ServiceException("msg", cause)`
 - **4.** Use try-with-resources for all AutoCloseable resources
 - **5.** Prefer unchecked exceptions for modern APIs
 - **6.** Include context in exception messages (IDs, values, state)
 - **7.** Don't use exceptions for control flow — they are expensive
 - **8.** Design custom exceptions with error codes and contextual fields
 - **9.** Log at the boundary, not at every layer (avoid double-logging)
 - **10.** Test exception handling: use assertThrows in unit tests
-